```
http.Handle("/metrics", promhttp.HandlerFor(
        registry,
        promhttp.HandlerOpts{
            EnableOpenMetrics: true,
        },
    ))
```

Now, we need to pull in all the dependencies, so (assuming you have Go version 1.20 or above installed) do the following:

```
go mod vendor
```

You should now see a subdirectory, `vendor/`, which has all the necessary packages available locally.

Next, let's go back to the main directory, `ch11/metrics-traces`, and bring up the Docker Compose stack defined in docker-compose.yaml:

```
docker-compose -f docker-compose.yaml up
```

The following listing shows the relevant snippets of the Docker Compose file we are using for the metrics–traces example.

Listing 11.1 The relevant snippets of the Docker Compose file for the correlation setup

```
prometheus:
    image: prom/prometheus:v2.33.4
    ports:
    - "9090:9090"
    volumes:
    - ./prometheus.yml:/etc/prometheus/prometheus.yml
    command:
    - "--config.file=/etc/prometheus/prometheus.yml"
    - "--storage.tsdb.path=/prometheus"
    - "--enable-feature=exemplar-storage"
  grafana:
    image: grafana/grafana:latest
    ports:
    - "3000:3000"
    volumes:
    - ./grafana-storage:/var/lib/grafana
    - ./grafana:/etc/grafana/provisioning
    - ./grafana/dashboards:/var/lib/grafana/dashboards
```

Need to configure Prometheus to support exemplars (in-memory)

Used for automatically configuring the Prometheus and Jaeger data sources as well as the dashboard

Let us move on to the OpenTelemetry collector configuration shown in the following listing—the otel-config.yaml file.

Listing 11.2 The OpenTelemetry collector configuration for the correlation setup

```
exporters:
  prometheus:
    endpoint: "0.0.0.0:1234"
    metric_expiration: 180m
```